cJSON JSON 字符串生成使用示例

基于 cJSON 1.7.19

内容：各种数据组合方式的构建、查找、修改、增加、删除。

本文所有示例均为完整可编译程序（含头文件引用与 main 函数），编译方式：

gcc 示例.c cJSON.c -o 示例

目录

1. 构建基本流程
2. 组合方式一：简单键值对象
3. 组合方式二：对象嵌套对象
4. 组合方式三：对象含数组
5. 组合方式四：对象数组
6. 组合方式五：多层混合嵌套
7. 组合方式六：从 C 数组快速生成
8. 组合方式七：Raw 片段嵌入
9. 内容的增加
10. 内容的查找
11. 内容的修改
12. 内容的删除
13. 完整综合示例
14. 内存释放注意事项

1. 构建基本流程

任何 JSON 生成都遵循固定四步：创建树 → 添加内容 → 序列化输出 → 释放内存。

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
  /* 第1步：创建根节点（对象或数组） */
  cJSON *root = cJSON_CreateObject();

  /* 第2步：添加内容 */
  cJSON_AddStringToObject(root, "hello", "world");

  /* 第3步：序列化为字符串
    cJSON_Print        格式化输出（带缩进换行）
    cJSON_PrintUnformatted 紧凑输出（无空白，适合传输） */
  char *out = cJSON_PrintUnformatted(root);
  printf("%s\n", out);       /* 输出: {"hello":"world"} */

  /* 第4步：释放内存
    注意：字符串用 free，cJSON 树用 cJSON_Delete，二者不可混用 */
  free(out);
  cJSON_Delete(root);
```

```
    return 0;
}
```

2. 组合方式一：简单键值对象

目标 JSON：

```
{"name":"Tom","age":20,"score":95.5,"is_vip":true,"remark":null}
```

完整代码：

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
    cJSON *root = cJSON_CreateObject();

    /* 一步到位添加各种基础类型字段 */
    cJSON_AddStringToObject(root, "name", "Tom");   /* 字符串 */
    cJSON_AddNumberToObject(root, "age", 20);       /* 整数 */
    cJSON_AddNumberToObject(root, "score", 95.5);   /* 浮点数 */
    cJSON_AddBoolToObject(root, "is_vip", 1);       /* 布尔：1=true 0=false */
    cJSON_AddNullToObject(root, "remark");          /* null 值 */

    /* 等价的分步写法（先创建节点再挂载）：
       cJSON_AddItemToObject(root, "name", cJSON_CreateString("Tom")); */

    char *out = cJSON_PrintUnformatted(root);
    printf("%s\n", out);
    free(out);
    cJSON_Delete(root);
    return 0;
}
```

3. 组合方式二：对象嵌套对象

目标 JSON：

```
{"user":{"name":"Tom","addr":{"city":"北京","zip":"100000"}},"level":3}
```

完整代码：

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
    cJSON *root = cJSON_CreateObject();

    /* cJSON_AddObjectToObject 会创建子对象并返回其指针，
       后续直接向该子对象添加字段，无需再次查找 */
    cJSON *user = cJSON_AddObjectToObject(root, "user");
    cJSON_AddStringToObject(user, "name", "Tom");

    /* 在 user 内再嵌套一层 addr 对象 */
    cJSON *addr = cJSON_AddObjectToObject(user, "addr");
    cJSON_AddStringToObject(addr, "city", "北京");
    cJSON_AddStringToObject(addr, "zip", "100000");

    /* 根节点的其他字段 */
```

```
    cJSON_AddNumberToObject(root, "level", 3);

    char *out = cJSON_PrintUnformatted(root);
    printf("%s\n", out);
    free(out);
    cJSON_Delete(root);
    return 0;
}
```

4. 组合方式三：对象含数组

目标 JSON：

```
{"tags":["c","python","go"],"scores":[90,85,92],"mix":[1,"two",true,null],"empty":[]}
```

完整代码：

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
    /* 数据源：C 数组先定义在循环外 */
    const char *tag_strs[] = {"c", "python", "go"};
    int        score_vals[] = {90, 85, 92};
    int i;

    cJSON *root = cJSON_CreateObject();

    /* cJSON_AddArrayToObject 创建子数组并返回其指针 */
    cJSON *tags = cJSON_AddArrayToObject(root, "tags");
    for (i = 0; i &lt; 3; i++) {
        cJSON_AddItemToArray(tags, cJSON_CreateString(tag_strs[i]));
    }

    /* 数字数组 */
    cJSON *scores = cJSON_AddArrayToObject(root, "scores");
    for (i = 0; i &lt; 3; i++) {
        cJSON_AddItemToArray(scores, cJSON_CreateNumber(score_vals[i]));
    }

    /* 混合类型数组：同一数组内可放任意类型元素 */
    cJSON *mix = cJSON_AddArrayToObject(root, "mix");
    cJSON_AddItemToArray(mix, cJSON_CreateNumber(1));
    cJSON_AddItemToArray(mix, cJSON_CreateString("two"));
    cJSON_AddItemToArray(mix, cJSON_CreateTrue());
    cJSON_AddItemToArray(mix, cJSON_CreateNull());

    /* 空数组 */
    cJSON_AddArrayToObject(root, "empty");

    char *out = cJSON_PrintUnformatted(root);
    printf("%s\n", out);
    free(out);
    cJSON_Delete(root);
    return 0;
}
```

5. 组合方式四：对象数组

目标 JSON（数组的每个元素是一个对象，实际项目最常见）：

{"users":[{"id":1,"name":"张三"},{"id":2,"name":"李四"}],"count":2}

完整代码：

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
    const char *names[] = {"张三", "李四"};
    int i;

    cJSON *root = cJSON_CreateObject();
    cJSON *users = cJSON_AddArrayToObject(root, "users");

    for (i = 0; i &lt; 2; i++) {
        /* 每个数组元素独立创建一个对象 */
        cJSON *u = cJSON_CreateObject();
        cJSON_AddNumberToObject(u, "id", i + 1);
        cJSON_AddStringToObject(u, "name", names[i]);
        /* 挂进数组后所有权转移，禁止再单独 cJSON_Delete(u) */
        cJSON_AddItemToArray(users, u);
    }
    cJSON_AddNumberToObject(root, "count", 2);

    char *out = cJSON_PrintUnformatted(root);
    printf("%s\n", out);
    free(out);
    cJSON_Delete(root);
    return 0;
}
```

6. 组合方式五：多层混合嵌套

目标 JSON：

{"status":"ok","code":0,"data":{"list":[{"id":1,"tags":["a","b"]},{"id":2,"tags":[]}],"page":{"cur
":1,"size":10},"extra":null}}

完整代码：

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
    /* 构建规律：自顶向下创建，拿到父指针后立即填充子内容 */
    cJSON *root = cJSON_CreateObject();
    cJSON_AddStringToObject(root, "status", "ok");
    cJSON_AddNumberToObject(root, "code", 0);

    /* data 层 */
    cJSON *data = cJSON_AddObjectToObject(root, "data");
    cJSON *list = cJSON_AddArrayToObject(data, "list");

    /* 数组元素1：对象内含字符串数组 */
    cJSON *e1 = cJSON_CreateObject();
    cJSON_AddNumberToObject(e1, "id", 1);
    cJSON *t1 = cJSON_AddArrayToObject(e1, "tags");
    cJSON_AddItemToArray(t1, cJSON_CreateString("a"));
    cJSON_AddItemToArray(t1, cJSON_CreateString("b"));
    cJSON_AddItemToArray(list, e1);
```

```
    /* 数组元素2：对象内含空数组 */
    cJSON *e2 = cJSON_CreateObject();
    cJSON_AddNumberToObject(e2, "id", 2);
    cJSON_AddArrayToObject(e2, "tags");
    cJSON_AddItemToArray(list, e2);

    /* data 下再放一个分页对象 */
    cJSON *page = cJSON_AddObjectToObject(data, "page");
    cJSON_AddNumberToObject(page, "cur", 1);
    cJSON_AddNumberToObject(page, "size", 10);

    cJSON_AddNullToObject(data, "extra");

    /* 用格式化打印观察层级结构 */
    char *out = cJSON_Print(root);
    printf("%s\n", out);
    free(out);
    cJSON_Delete(root);
    return 0;
}
```

7. 组合方式六：从 C 数组快速生成

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
    /* 各类 C 数组数据源 */
    int         ids[]   = {1, 2, 3, 4};
    float       temps[] = {36.5f, 37.0f, 36.8f};
    double      prices[] = {9.9, 19.9, 29.9};
    const char *names[] = {"apple", "banana", "cherry"};

    cJSON *root = cJSON_CreateObject();

    /* CreateXxxArray 一步把整个 C 数组转成 JSON 数组并挂载 */
    cJSON_AddItemToObject(root, "ids",    cJSON_CreateIntArray(ids, 4));
    cJSON_AddItemToObject(root, "temps",  cJSON_CreateFloatArray(temps, 3));
    cJSON_AddItemToObject(root, "prices", cJSON_CreateDoubleArray(prices, 3));
    cJSON_AddItemToObject(root, "names",  cJSON_CreateStringArray(names, 3));

    char *out = cJSON_PrintUnformatted(root);
    printf("%s\n", out);
    /* 输出: {"ids":[1,2,3,4],"temps":[36.5,37,36.8],
          "prices":[9.9,19.9,29.9],"names":["apple","banana","cherry"]} */
    free(out);
    cJSON_Delete(root);
    return 0;
}
```

8. 组合方式七：Raw 片段嵌入

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
    cJSON *root = cJSON_CreateObject();
    cJSON_AddStringToObject(root, "type", "raw_demo");

    /* Raw 内容原样输出、不做转义，适合嵌入已有的 JSON 文本。
```

```
    注意：内容必须自身是合法 JSON，否则会破坏整体结构 */
   cJSON_AddRawToObject(root, "payload", "{\"already\":\"json\"}");

   char *out = cJSON_PrintUnformatted(root);
   printf("%s\n", out);
   /* 输出: {"type":"raw_demo","payload":{"already":"json"}} */
   free(out);
   cJSON_Delete(root);
   return 0;
}
```

9. 内容的增加

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
   /* 先构造基础数据: {"arr":[1,3],"shared":"共享数据"} */
   cJSON *root = cJSON_CreateObject();
   cJSON *arr = cJSON_AddArrayToObject(root, "arr");
   cJSON_AddItemToArray(arr, cJSON_CreateNumber(1));
   cJSON_AddItemToArray(arr, cJSON_CreateNumber(3));

   /* 1) 对象尾部增加字段 */
   cJSON_AddNumberToObject(root, "new_field", 123);

   /* 2) 数组尾部追加元素 */
   cJSON_AddItemToArray(arr, cJSON_CreateNumber(4));

   /* 3) 数组指定位置插入（索引1处，原元素右移）→ arr 变为 [1,2,3,4] */
   cJSON_InsertItemInArray(arr, 1, cJSON_CreateNumber(2));

   /* 4) 引用方式添加：同一节点挂到多处，不产生拷贝。
     注意被引用节点 shared 必须先于引用方存在的更久 */
   cJSON *shared = cJSON_CreateString("共享数据");
   cJSON_AddItemToObject(root, "origin", shared);
   cJSON_AddItemReferenceToObject(root, "alias", shared);

   char *out = cJSON_PrintUnformatted(root);
   printf("%s\n", out);
   /* 输出: {"arr":[1,2,3,4],"new_field":123,
         "origin":"共享数据","alias":"共享数据"} */
   free(out);
   cJSON_Delete(root);
   return 0;
}
```

10. 内容的查找

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
   /* 测试数据: {"data":{"page":{"cur":1}},"users":[{"id":1,"name":"张三"},
                                    {"id":2,"name":"李四"}]} */
   const char *json =
      "{\"data\":{\"page\":{\"cur\":1}},"
      "\"users\":[{\"id\":1,\"name\":\"张三\"},{\"id\":2,\"name\":\"李四\"}]}";
   cJSON *root = cJSON_Parse(json);
```

```
    if (root == NULL) {
        printf("解析失败: %s\n", cJSON_GetErrorPtr());
        return 1;
    }

    /* 1) 按键查找：推荐区分大小写版本，避免意外命中。
      cJSON_GetObjectItem 不区分大小写，"Name"/"NAME" 都会命中 */
    if (cJSON_HasObjectItem(root, "data")) {
        printf("存在 data 字段\n");
    }

    /* 2) 嵌套路径逐层查找（每层判空防崩溃） */
    cJSON *data = cJSON_GetObjectItem(root, "data");
    cJSON *page = data ? cJSON_GetObjectItem(data, "page") : NULL;
    cJSON *cur  = page ? cJSON_GetObjectItem(page, "cur") : NULL;
    if (cJSON_IsNumber(cur)) {
        printf("data.page.cur = %g\n", cur-&gt;valuedouble);
    }

    /* 3) 数组按索引访问 + 遍历两种方式 */
    cJSON *users = cJSON_GetObjectItem(root, "users");
    if (cJSON_IsArray(users)) {
        /* 方式一：索引 */
        int n = cJSON_GetArraySize(users);
        for (int i = 0; i &lt; n; i++) {
            cJSON *u = cJSON_GetArrayItem(users, i);
            cJSON *name = cJSON_GetObjectItem(u, "name");
            if (cJSON_IsString(name)) printf("[%d] %s\n", i, name-&gt;valuestring);
        }
        /* 方式二：cJSON_ArrayForEach 宏（对象/数组通用） */
        cJSON *elem = NULL;
        cJSON_ArrayForEach(elem, users) {
            cJSON *id = cJSON_GetObjectItem(elem, "id");
            if (cJSON_IsNumber(id)) printf("id = %g\n", id-&gt;valuedouble);
        }
    }

    cJSON_Delete(root);
    return 0;
}
```

11. 内容的修改

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
    /* 测试数据 */
    const char *json =
        "{\"name\":\"Tom\",\"age\":20,\"is_vip\":true,\"arr\":[1,2,3]}";
    cJSON *root = cJSON_Parse(json);
    if (root == NULL) return 1;

    /* 1) 修改数字值（类型必须匹配） */
    cJSON *age = cJSON_GetObjectItem(root, "age");
    if (cJSON_IsNumber(age)) cJSON_SetNumberValue(age, 99);

    /* 2) 修改字符串值 */
    cJSON *name = cJSON_GetObjectItem(root, "name");
    if (cJSON_IsString(name)) cJSON_SetValuestring(name, "新名字");

    /* 3) 修改布尔值（宏，仅对布尔节点生效） */
    cJSON *vip = cJSON_GetObjectItem(root, "is_vip");
    if (cJSON_IsBool(vip)) cJSON_SetBoolValue(vip, 0);  /* 改为 false */
```

```
    /* 4) 整体替换（允许换类型）："age":99 → "age":"九十九"
       注意：Replace 会自动 Delete 旧节点并拷贝键名，旧指针失效 */
    cJSON_ReplaceItemInObjectCaseSensitive(root, "age",
                                           cJSON_CreateString("九十九"));

    /* 5) 替换数组第 0 个元素：arr[0] 由 1 变为 -1 */
    cJSON *arr = cJSON_GetObjectItem(root, "arr");
    if (cJSON_IsArray(arr)) {
        cJSON_ReplaceItemInArray(arr, 0, cJSON_CreateNumber(-1));
    }

    char *out = cJSON_PrintUnformatted(root);
    printf("%s\n", out);
    /* 输出: {"name":"新名字","age":"九十九","is_vip":false,"arr":[-1,2,3]} */
    free(out);
    cJSON_Delete(root);
    return 0;
}
```

12. 内容的删除

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
    /* 测试数据 */
    const char *json =
        "{\"keep\":1,\"remark\":\"x\",\"data\":{\"v\":2},\"arr\":[1,2,3]}";
    cJSON *root = cJSON_Parse(json);
    if (root == NULL) return 1;

    /* 1) Delete：从父节点移除并立即释放内存 */
    cJSON_DeleteItemFromObject(root, "remark");  /* 删对象字段 */

    cJSON *arr = cJSON_GetObjectItem(root, "arr");
    if (cJSON_IsArray(arr)) {
        cJSON_DeleteItemFromArray(arr, 0);       /* 删数组第 0 个元素 */
    }

    /* 2) Detach：只摘除链接、不释放内存，节点可挂到别处。
       Delete 与 Detach 的区别：前者释放，后者内存归调用者 */
    cJSON *moved = cJSON_DetachItemFromObject(root, "data");
    cJSON *backup = cJSON_CreateObject();
    cJSON_AddItemToObject(backup, "backup", moved);  /* 挂到新树 */
    /* 若不挂到别处，必须自行 cJSON_Delete(moved) 防泄漏 */

    char *out = cJSON_PrintUnformatted(root);
    printf("root:   %s\n", out);     /* {"keep":1,"arr":[2,3]} */
    free(out);

    out = cJSON_PrintUnformatted(backup);
    printf("backup: %s\n", out);     /* {"backup":{"v":2}} */
    free(out);

    cJSON_Delete(root);
    cJSON_Delete(backup);
    return 0;
}
```

13. 完整综合示例

演示完整业务闭环：构建用户列表 → 查找 → 修改 → 增加 → 删除 → 输出。

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include "cJSON.h"

int main(void)
{
    /* ---------- 1. 构建 ---------- */
    cJSON *root = cJSON_CreateObject();
    cJSON_AddStringToObject(root, "module", "user_mgr");

    cJSON *users = cJSON_AddArrayToObject(root, "users");
    const char *names[] = {"张三", "李四", "王五"};
    for (int i = 0; i &lt; 3; i++) {
        cJSON *u = cJSON_CreateObject();
        cJSON_AddNumberToObject(u, "id", i + 1);
        cJSON_AddStringToObject(u, "name", names[i]);
        cJSON_AddBoolToObject(u, "active", 1);
        cJSON_AddItemToArray(users, u);    /* 挂载后 u 归数组所有 */
    }

    /* ---------- 2. 查找：找 id==2 的用户 ---------- */
    cJSON *target = NULL;
    cJSON *elem = NULL;
    cJSON_ArrayForEach(elem, users) {
        cJSON *id = cJSON_GetObjectItem(elem, "id");
        if (cJSON_IsNumber(id) &amp;&amp; (int)id-&gt;valuedouble == 2) {
            target = elem;              /* 找到即记录指针 */
        }
    }

    /* ---------- 3. 修改：改名 + 禁用 ---------- */
    if (target != NULL) {
        cJSON_SetValuestring(cJSON_GetObjectItem(target, "name"), "李四-改");
        cJSON_SetBoolValue(cJSON_GetObjectItem(target, "active"), 0);
    }

    /* ---------- 4. 增加：追加一个新用户 ---------- */
    cJSON *nu = cJSON_CreateObject();
    cJSON_AddNumberToObject(nu, "id", 4);
    cJSON_AddStringToObject(nu, "name", "赵六");
    cJSON_AddItemToArray(users, nu);

    /* ---------- 5. 删除：删掉第 0 个用户（id==1） ---------- */
    cJSON_DeleteItemFromArray(users, 0);

    /* ---------- 6. 输出 ---------- */
    char *out = cJSON_Print(root);
    printf("%s\n", out);
    free(out);

    cJSON_Delete(root);  /* 整棵树一次回收 */
    return 0;
}
```

输出：

```
{
    "module": "user_mgr",
    "users": [{
            "id": 2,
            "name": "李四-改",
            "active": false
        }, {
            "id": 3,
            "name": "王五",
```

```
        "active": true
    }, {
        "id": 4,
        "name": "赵六"
    }]
}
```

14. 内存释放注意事项

操作	规则
AddItem 之后	子节点归父节点，禁止单独 cJSON_Delete 子节点
cJSON_Print* 返回的字符串	用 free() 释放，不是 cJSON_Delete
Detach 摘下的节点	归调用者，用 cJSON_Delete 释放或挂到别处
Replace 之后	旧节点已被自动释放，旧指针失效
引用添加（AddItemReference）	只创建引用壳，被引用节点须比引用方活得更久
整棵树	最后一次 cJSON_Delete(root) 全部回收